

Woxsen University

School of Technology



B.Tech – Computer Science and Engineering

PROJECT BASED LEARNING REPORT

Title of the Project

(Design, Implementation and Performance Analysis of an AI-Based Student Timetable Generator)

Course Code: _____

Course Name: Fundamentals of AI (FOAI)

Semester: _____

Academic Year: 2026-27

Submitted by:

1. Y.Nikhil – 2306

2. Y.Rupesh– 2305

3. V.Sailesh – 2301

4. V.Sathvik– 2302

Section: _____

Under the Guidance of:

Faculty Name: _____

Designation: _____

CERTIFICATE

This is to certify that the Project titled "AI-Based Student Timetable Generator" is a bonafide work carried out by the following students of B.Tech – Computer Science and Engineering during the Academic Year 2026–27 under my supervision and guidance.

Yarlagadda Nikhil – 25WU0102306

Yallamgari Rupesh – 25WU0102305

V Sailesh – 25WU0102301

V.Sathvik – 25WU0102302

Guide Signature: _____

Name of the Guide: _____

Designation: _____

DECLARATION

We hereby declare that the Project titled "AI Based Student Timetable Generator" submitted by us is an original work carried out under the guidance of the faculty mentioned above. We confirm that this work has not been submitted in part or full for the award of any degree, diploma, or other academic qualification in this or any other institution.

All sources of information used in this project have been properly acknowledged and cited as per university referencing standards. We further declare that the plagiarism percentage is within the permissible limit prescribed by the institution.

Place: _____

Date: _____

Signatures of the Candidates:

1. Y Nikhil – 25WU0102306
2. Y. Rupesh – 25WU0102305
3. V.Sailesh – 25WU0102301
4. V.Sathvik – 25WU0102302

ACKNOWLEDGEMENT

We express our sincere gratitude to Woxsen University and the School of Technology for providing the academic environment, laboratory facilities and resources required to carry out this project.

We are deeply thankful to our faculty guide for the valuable guidance, constructive criticism and encouragement provided throughout the work. The guidance helped us understand the scheduling problem as an AI based constraint-optimization task and supported us in developing the proposed timetable generation system.

We also thank the Head of the Department of Computer Science and Engineering and all faculty members for their support. Finally, we thank our parents, friends and team members for their continuous cooperation and motivation during the development and documentation of this project.

1. Y. Nikhil – 25WU0102306
2. Y. Rupesh – 25WU0102305
3. V.Sailesh – 25WU0102301
4. V.Sathvik – 25WU0102302

ABSTRACT

Timetable preparation is a complex scheduling task because subjects, teachers, rooms, student groups and time slots must be coordinated while satisfying several rules at the same time. Traditional manual preparation can take days, may create double-booking conflicts and becomes difficult to modify when even one constraint changes. This project presents an AI-Based Student Timetable Generator that treats timetable construction as a constraint-optimization problem.

The proposed system accepts subjects, teachers, rooms, time slots and student groups as inputs. It separates hard constraints, such as preventing teacher, room or class clashes, from soft preferences, such as spreading workload and free periods more evenly. An AI scheduling engine searches for a suitable arrangement, validates the generated schedule, and improves or regenerates it when the inputs or constraints change.

A web-based interface is proposed to make the system accessible to students and staff. Users can enter scheduling information, generate a timetable, view it by class, teacher or day, edit or regenerate it, and download the final result. The presentation also highlights automatic clash detection, sensible placement of lectures and labs, holiday and examination-day handling, and quick regeneration after changes.

The project demonstrates how AI-based scheduling can reduce repetitive administrative work, improve consistency, and provide a predictable weekly routine for students and teachers. The current work can be extended with electives and multi-section support, mobile notifications, and learning from previous scheduling preferences.

TABLE OF CONTENTS

1. INTRODUCTION	9
1.1 Background	9
1.2 Motivation and Relevance to Real-World Problems	9
1.3 Scope and Organisation of the Report	10
2. PROBLEM STATEMENT AND OBJECTIVES	11
2.1 Problem Definition	11
2.2 Objectives	11
2.3 Constraints and Assumptions	12
3. LITERATURE REVIEW	13
3.1 Automated Timetabling	13
3.2 Timetabling as a Constraint Satisfaction Problem	13
3.3 Heuristic and Evolutionary Approaches	13
3.4 Local Search and Optimization	13
3.5 Web-Based Scheduling Systems	14
3.6 AI-Assisted Scheduling and Adaptation	14
3.7 Comparative Summary	14
4. METHODOLOGY AND SYSTEM DESIGN	15
4.1 Overview of the Approach	15
4.2 Overall System Architecture	15
4.3 Flowchart of the Scheduling Process	16
4.4 AI Scheduling and Constraint Model	17
4.5 Data Representation	17
4.6 Tools and Technologies	17
4.7 Use-Case and Data-Flow Diagrams	18
5. IMPLEMENTATION	20
5.1 Input Data Description	20
5.2 Module-wise Implementation	20
5.3 Pseudo Code of the Main Program	20
5.4 Website Interface and Output	21
5.5 Testing	21
5.6 Regeneration and Error Handling	22
6. RESULTS AND DISCUSSION	23
6.1 Evaluation Setup	23
6.2 Functional Results	23
6.3 Interpretation of Results	23
6.4 Comparison with the Objectives	24
7. CONCLUSION AND FUTURE SCOPE	25
7.1 Summary	25
7.2 Learning Outcomes	25
7.3 Future Enhancements	25
REFERENCES	27

LIST OF FIGURES

Fig. 4.1: Overall system architecture	15
Fig. 4.2: Flowchart of the timetable generation process	16
Fig. 4.4: Use-case diagram of the AI timetable generator	18
Fig. 4.5: Level-1 data-flow diagram	19
Fig. 5.1: Project website / AI scheduling interface	21

LIST OF TABLES

Table 1.1: Organisation of the report	10
Table 2.1: Objectives and success criteria	11
Table 2.2: Functional and non-functional requirements	12
Table 2.3: Constraints and assumptions	12
Table 3.1: Comparison of related approaches	14
Table 4.1: Scheduling entities and their roles	17
Table 4.2: Hard and soft constraints	17
Table 4.3: Tools and technologies used	17
Table 5.1: Input data fields	20
Table 5.2: Main system functions	20
Table 5.3: Test cases and expected behaviour	22
Table 6.1: Functional evaluation summary	23
Table 6.2: Objective-wise result comparison	24

1. INTRODUCTION

1.1 Background

A timetable is a structured arrangement that determines when classes, teachers, rooms and student groups are scheduled. Although the final timetable appears simple, preparing it is a constrained scheduling problem. A valid schedule must place every required class into a suitable time slot while avoiding conflicts between shared resources.

In many institutions, timetable preparation is still handled manually using spreadsheets or repeated trial-and-error adjustments. This approach becomes increasingly difficult when the number of subjects, teachers, rooms, sections and time slots grows. A small change, such as moving one lecture, may affect several other allocations and force the scheduler to repeat the process.

The proposed AI-Based Student Timetable Generator approaches the problem as a constraint-optimization task. The project presentation identifies subjects, teachers, rooms, time slots and student groups as the primary inputs. The system separates hard constraints, which should not be violated, from soft constraints, which describe desirable properties such as balanced workload.

1.2 Motivation and Relevance to Real World Problems

The main motivation is to reduce the time and repeated effort involved in timetable preparation. The project identifies four practical problems: manual scheduling takes days, teachers or rooms can be double-booked, small changes can require major rework, and poorly distributed classes can create an uneven workload.

From an AI perspective, the problem is useful because it requires the system to search through many possible arrangements and select one that satisfies important constraints while improving softer preferences. This is similar to other optimization problems where a system must choose a good configuration from a large number of possible configurations.

The project is also relevant to educational administration. A timetable generator can reduce repetitive work for staff and give students a clearer and more predictable weekly schedule. The proposed website further reduces the technical barrier by allowing users to enter data, generate a schedule, view it in different ways, and regenerate it when requirements change.

1.3 Scope and Organisation of the Report

The scope covers the design of an AI based timetable generation system, the modelling of timetable inputs and constraints, the generation and validation of candidate schedules, and the presentation of the resulting timetable through a website. The presentation specifically includes class, teacher, day, room and time-slot handling, automatic clash detection, regeneration after changes, and download-oriented output.

The present report does not claim a specific AI algorithm that is not identified in the project presentation. Instead, the scheduling engine is described at the level supported by the project: constraint modelling, efficient search, candidate scoring, validation and regeneration. Exact implementation details such as a particular optimization library or programming framework can be added later if they are part of the final code.

Chapter	Title	Content
1	Introduction	Background, motivation, real-world relevance, scope and organisation.
2	Problem Statement and Objectives	Problem definition, objectives, requirements, constraints and assumptions.
3	Literature Review	Related work on automated timetabling, constraint satisfaction and optimization.
4	Methodology and System Design	Architecture, workflow, constraints, data representation, tools, use-case and data-flow diagrams.
5	Implementation	Input model, modules, pseudocode, website output, testing and error handling.
6	Results and Discussion	Functional evaluation, interpretation and comparison with project objectives.
7	Conclusion and Future Scope	Summary, learning outcomes and future enhancements.
References	References	Sources used for the report and background study.

Table 1.1: Organisation of the report

2. PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem Definition

The problem addressed by this project is the automatic generation of a student timetable that satisfies required scheduling constraints and produces a practical weekly arrangement. The system must coordinate subjects, teachers, rooms, time slots and student groups without creating unacceptable overlaps.

A timetable is considered invalid when a teacher is assigned to two classes at the same time, a room is used by two classes simultaneously, or a student group is expected to attend more than one class in the same period. In addition to these hard constraints, the project aims to improve softer preferences such as distributing subjects across the week and avoiding an overloaded day.

The proposed solution therefore treats timetable generation as a constraint-optimization problem. Candidate arrangements are generated and checked, conflicting arrangements are rejected or penalized, and the system continues searching or regenerating until a suitable timetable is produced. The project presentation summarizes the process as: Input → Constraint model → AI engine → Validation → Website output.

2.2 Objectives

No.	Objective	Success Criterion
O1	Automate timetable generation	Generate a timetable automatically from subjects, teachers, rooms, slots and student groups.
O2	Avoid clashes	Prevent teacher, room and student-group overlaps.
O3	Save time	Reduce the repeated manual effort required for timetable preparation.
O4	Balance workload	Spread subjects and free periods across the week in a more practical way.
O5	Support flexible updates	Allow quick regeneration when a constraint or input changes.
O6	Provide easy access	Offer a simple website through which students and staff can view and manage the generated schedule.

Table 2.1: Objectives and success criteria

2.3 Constraints and Assumptions

Type	Requirement
Functional	Accept subjects, teachers, rooms, time slots and student groups as inputs.
Functional	Represent hard constraints and soft scheduling preferences separately.
Functional	Generate a timetable automatically using the AI scheduling engine.
Functional	Check teacher, room and student-group clashes.
Functional	Display the schedule by class, teacher or day.
Functional	Allow editing and quick regeneration after changes.
Functional	Provide an option to download or export the generated timetable.
Non-functional	Produce results quickly enough for practical interactive use.
Non-functional	Provide clear validation and error messages for incomplete or invalid input.
Non-functional	Keep the scheduling logic separated from the website presentation layer.

Table 2.2: Functional and non-functional requirements

Category	Description
Constraint	Teacher must not be double-booked.
Constraint	Room must not be double-booked.
Constraint	A student group must not have overlapping classes.
Constraint	Required lectures and labs must be placed in available periods.
Preference	Subjects should be distributed across the week.
Preference	Hard or demanding subjects should not be unnecessarily stacked on one day.
Preference	Breaks and free periods should be placed sensibly.
Assumption	All required scheduling information is provided by the user.
Assumption	The set of rooms, teachers and time slots is known before generation.
Assumption	The final timetable can be regenerated if constraints or inputs change.

Table 2.3: Constraints and assumptions

3. LITERATURE REVIEW

Automated timetabling has been studied as a difficult combinatorial optimization problem because a large number of possible schedules can be formed even for a moderate institution. Research in this area generally models timetable generation using constraints and then applies exact methods, heuristics, local search, evolutionary methods or hybrid techniques to find acceptable schedules.

3.1 Automated Timetabling

Early work on educational timetabling established that the task involves assigning events to periods and resources while satisfying institutional rules. The main challenge is that the number of possible assignments grows rapidly as the number of classes and resources increases. This makes a simple exhaustive search impractical for realistic institutions.

3.2 Timetabling as a Constraint Satisfaction Problem

Constraint satisfaction provides a natural mathematical model for timetabling. Variables can represent scheduled classes, domains can represent possible periods and rooms, and constraints can prevent invalid combinations. Hard constraints define feasibility, while soft constraints can be represented as preferences or penalties. This distinction is directly reflected in the proposed project, which separates hard rules from soft preferences before the AI scheduling stage.

3.3 Heuristic and Evolutionary Approaches

Because realistic timetable spaces are large, heuristic methods are often used to guide the search toward promising regions rather than examining every possibility. Evolutionary approaches such as genetic algorithms represent schedules as candidate solutions and improve them through selection and variation. Their main advantage is flexibility in handling multiple objectives, although the quality of the result depends on representation, fitness design and parameter selection.

3.4 Local Search and Optimization

Local-search methods begin with a candidate timetable and repeatedly make changes intended to reduce conflicts or improve the overall score. Examples include moving a class to another slot, exchanging two assignments, or changing a room. These methods can be efficient for large spaces, but they can also become trapped in locally good solutions unless diversification or restart strategies are used.

3.5 Web Based Scheduling Systems

Modern scheduling systems increasingly use web interfaces because timetable data is naturally shared among administrators, teachers and students. A web interface also makes regeneration and display easier: users can submit inputs, view the result by different dimensions and obtain an updated schedule without manually rebuilding spreadsheets.

3.6 AI Assisted Scheduling and Adaptation

An important direction in intelligent scheduling is rapid adaptation. The project specifically proposes automatic regeneration when an input or rule changes. This is valuable because real timetables are rarely fixed permanently: rooms can become unavailable, faculty assignments can change, and academic events can require modifications.

3.7 Comparative Summary

Approach	Main Idea	Strength	Limitation	Relation to This Project
Manual scheduling	Human assigns classes using repeated adjustments	Simple for very small cases	Slow and error-prone at scale	Problem being addressed
Constraint satisfaction	Represent variables, domains and constraints	Clear representation of feasibility	Can become computationally difficult	Core modelling idea
Heuristic search	Use rules to guide the search	Faster than exhaustive enumeration	May not find the best arrangement	Relevant to efficient search
Genetic algorithms	Evolve candidate schedules using a fitness score	Handles complex objective combinations	Parameter and encoding dependent	Related optimization approach
Local search	Improve an existing timetable through changes	Practical for large search spaces	Can get stuck in local optima	Useful for iterative improvement
Proposed system	AI scheduling with hard/soft constraints, validation and regeneration	Fast, flexible and user-oriented	Final performance depends on implementation	Direct project approach

Table 3.1: Comparison of related approaches

4. METHODOLOGY AND SYSTEM DESIGN

4.1 Overview of the Approach

The project follows a five-stage methodology taken directly from the project presentation: collect data, set constraints, perform AI scheduling, optimize and validate the timetable, and publish the result on the website. The data collection stage gathers subjects, teachers, rooms, time slots and student groups. The constraint stage separates hard rules from soft preferences. The AI engine then assigns classes to available periods, after which the result is checked and improved.

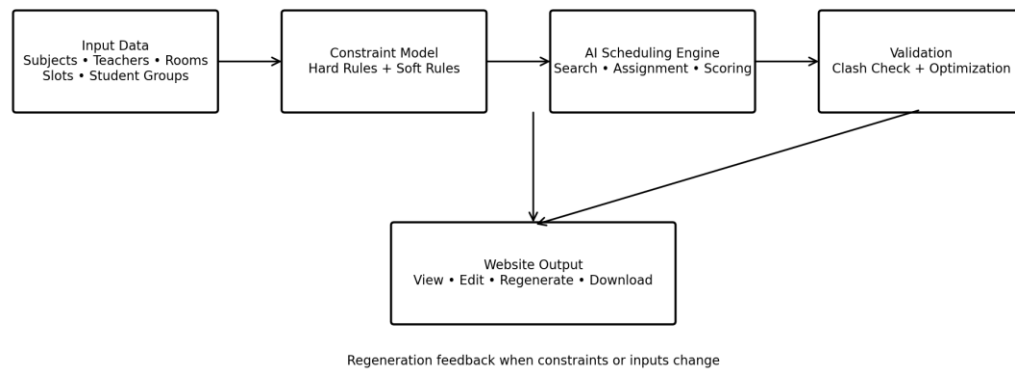


Fig. 4.1: Overall system architecture

4.2 Overall System Architecture

The architecture is divided into five logical layers. The Input Layer receives the timetable information. The Constraint Model converts institutional rules into checks and preferences. The AI Scheduling Engine searches for a suitable allocation. The Validation Layer checks conflicts and the quality of the candidate schedule. Finally, the Website Output Layer presents the timetable and provides interaction such as viewing, editing, regeneration and download.

This separation improves maintainability. Changes to the user interface do not need to change the constraint model, and changes to scheduling logic can be made without redesigning the output screens. The project presentation similarly describes the website as a place to enter data, generate a timetable, view it by class, teacher or day, and edit, regenerate or download it.

4.3 Flowchart of the Scheduling Process

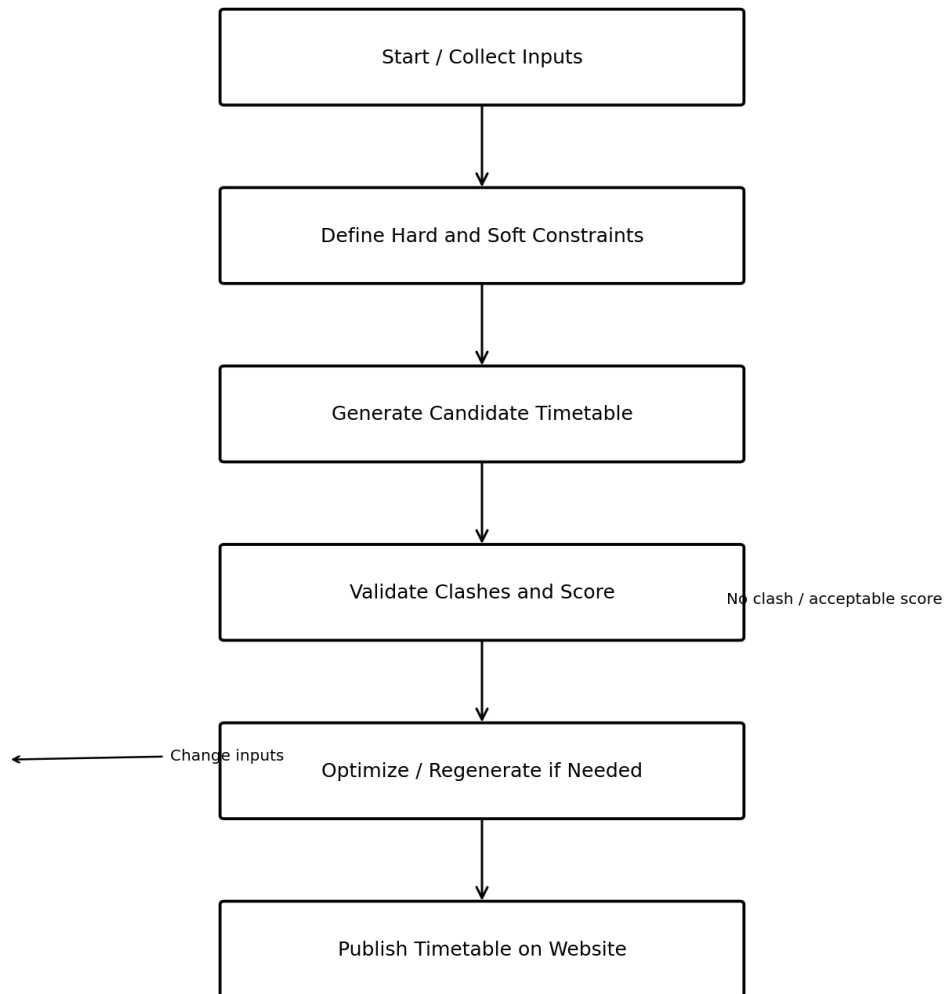


Fig. 4.2: Flowchart of the timetable generation process

The scheduling process begins when the user provides the required data. The system first validates that the information is usable. It then creates or evaluates candidate allocations under the hard constraints. A candidate with a conflict is rejected or penalized, while a feasible candidate is evaluated against softer preferences. If the result is not satisfactory, the system continues searching or regenerates the schedule. Once validation is successful, the timetable is sent to the website for display.

4.4 AI Scheduling and Constraint Model

The core of the project is the distinction between hard and soft constraints. Hard constraints represent conditions that should not be violated. Soft constraints represent desirable characteristics that improve the quality of a timetable but may be traded against other preferences.

Constraint Class	Examples	Treatment
Hard	No teacher double-booking	Reject or penalize any conflicting assignment.
Hard	No room double-booking	Reject or penalize simultaneous room use.
Hard	No student-group overlap	Prevent two classes for the same group in one period.
Hard	Valid lecture/lab period	Assign only to compatible available slots.
Soft	Balanced weekly workload	Prefer schedules that distribute subjects.
Soft	Avoid overloaded days	Penalize excessive concentration of classes.
Soft	Sensible breaks/free periods	Prefer practical spacing between sessions.

Table 4.2: Hard and soft constraints

A conceptual timetable score can be represented as a combination of feasibility and preference quality. One simple formulation is $\text{Score} = \text{HardPenalty} + \text{SoftPenalty}$, where a valid timetable has zero hard-constraint violations and the optimization stage attempts to minimize the remaining soft-constraint penalty. The exact numerical scoring formula is not specified in the project presentation and should therefore be treated as an implementation choice.

4.5 Data Representation

Entity	Important Information	Purpose
Subject	Subject name, type, required periods	Identifies what must be scheduled.
Teacher	Teacher identity and availability	Prevents double-booking and supports allocation.
Room	Room identity, type and availability	Assigns a suitable physical resource.
Time Slot	Day, period and availability	Defines when a class can occur.
Student Group	Class/section identity and requirements	Prevents student-group overlap.
Constraint	Rule or preference	Controls validity and schedule quality.
Timetable Entry	Subject + teacher + room + group + slot	Represents one scheduled class.

Table 4.1: Scheduling entities and their roles

4.6 Tools and Technologies

Tool / Technology	Role in the Project
AI scheduling logic	Generates and improves timetable arrangements under constraints.
Web interface	Accepts inputs and presents timetable results.
Validation logic	Checks input quality and scheduling conflicts.
Data representation	Stores subjects, teachers, rooms, groups and slots.
Export / download feature	Allows the generated schedule to be obtained by users.

Table 4.3: Tools and technologies used

The project presentation identifies the solution as a website but does not specify a particular programming language, web framework or AI library. Those implementation details have intentionally not been invented in this report.

4.7 Use-Case and Data-Flow Diagrams

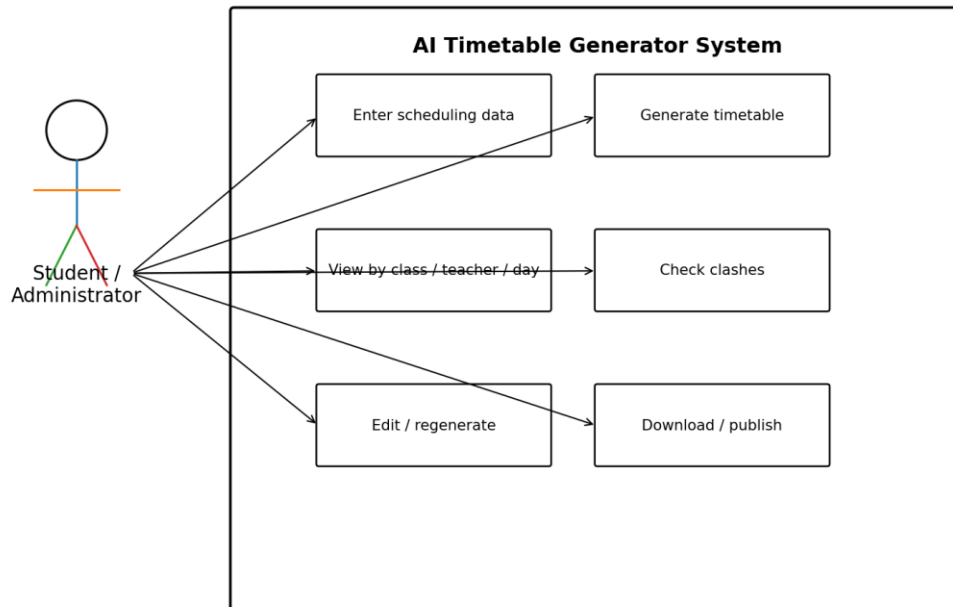


Fig. 4.4: Use-case diagram of the AI timetable generator

The use-case model shows the main interactions supported by the proposed system. A student or administrator can enter scheduling data, generate a timetable, view it by class, teacher or day,

check clashes, edit or regenerate the schedule, and download the result. These interactions match the website features presented in the project material.

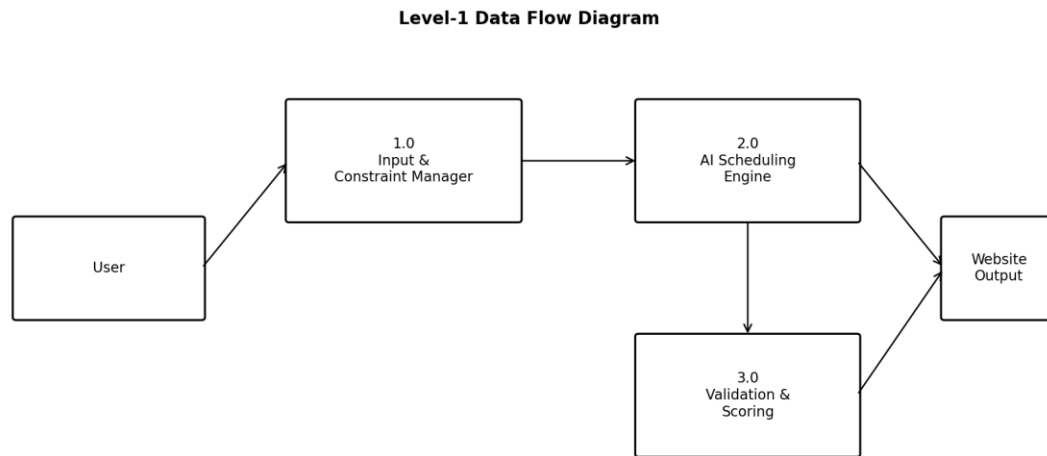


Fig. 4.5: Level-1 data-flow diagram

The data-flow diagram separates input management, AI scheduling, validation/scoring and website output. This logical separation reflects the project workflow and makes it easier to identify where a conflict or invalid input originates.

5. IMPLEMENTATION

5.1 Input Data Description

The implementation begins with the data required to construct a timetable. The project presentation explicitly identifies subjects, teachers, rooms, time slots and student groups as the main inputs. These inputs are converted into a constraint model before scheduling begins.

Field	Example	Validation / Use
Subject	Mathematics	Must be a valid scheduled subject.
Teacher	Faculty A	Must be available for the assigned slot.
Room	Room 204	Must be available and suitable.
Student Group	CSE-A	Must not receive two classes in one period.
Day	Monday	Used for weekly distribution.
Period	10:00–11:00	Must be an available slot.
Class Type	Lecture / Lab	Used when matching room and slot requirements.

Table 5.1: Input data fields

5.2 Module-wise Implementation

The system can be organised into five modules: Input and Validation, Constraint Manager, AI Scheduling Engine, Timetable Validator, and Website Interface. This modular design follows the same separation-of-concerns principle used throughout the project presentation.

Module	Main Responsibility
Input and Validation	Collect user data and identify missing or invalid fields.
Constraint Manager	Store hard constraints and soft scheduling preferences.
AI Scheduling Engine	Generate candidate assignments and search for a suitable timetable.
Timetable Validator	Check teacher, room and student-group clashes and evaluate schedule quality.
Website Interface	Display, edit, regenerate and download the timetable.

Table 5.2: Main system functions

5.3 Website Interface and Output

The project website is designed around simple actions. The presentation shows four primary capabilities: entering subjects, teachers, rooms and time slots; generating a timetable with one click; viewing the schedule by class, teacher or day; and editing, regenerating or downloading the result.

AI SCHEDULER / 01

Orbit

Schedule

A focused study plan,
generated around the rules
that matter.

SOLVER ENGINE

CSP SEARCH

HEURISTICS

MRV + FORWARD CHECKING

OBJECTIVE

BALANCED WEEK

YOUR STUDY PLAN, OPTIMIZED

Find the rhythm
for your week.

Add your subjects and preferences. The solver creates a clash-free timetable that keeps your week balanced.

AI METHOD

Constraint Satisfaction

Searches valid schedules,
then ranks the best fit.

SUBJECTS

5

subjects ready to schedule

SESSIONS

12

classes this week

SEARCH NODES

0

last generation

FIT SCORE

—

generate to calculate

CONFIGURE THE INPUTS

Subjects & preferences

+ ADD SUBJECT

PROGRAM / CLASS

B.Tech in AIML — Semester 3

WEEK STARTS

Monday

CLASSES / DAY

5 slots

01

SUBJECT NAME

Data Structures

FACULTY / TEACHER

Dr. Ananya Iyer

SESSIONS / WEEK

3

PREFERRED TIME

Morning

UNAVAILABLE DAY

Friday

02

SUBJECT NAME

Database Systems

FACULTY / TEACHER

Prof. K. Sharma

SESSIONS / WEEK

2

PREFERRED TIME

Midday

UNAVAILABLE DAY

No restriction

03

SUBJECT NAME

Operating Systems

FACULTY / TEACHER

Dr. R. Mehta

SESSIONS / WEEK

3

PREFERRED TIME

Morning

UNAVAILABLE DAY

Wednesday

04

SUBJECT NAME

Computer Networks

FACULTY / TEACHER

Prof. S. Nair

SESSIONS / WEEK

2

PREFERRED TIME

Afternoon

UNAVAILABLE DAY

No restriction

Fig. 5.1: Project presentation view showing how AI supports timetable generation on the website

The website concept also includes automatic handling of dates by skipping holidays and examination days, assignment of available teachers and suitable rooms, sensible placement of lectures and labs, and automatic regeneration when an input or rule changes.

5.4 Testing

Test ID	Test Description	Expected Behaviour	Status
T1	Enter valid subjects, teachers, rooms and slots	Input accepted	Pass / to be verified in final implementation
T2	Assign one teacher to two classes at the same time	Conflict detected or assignment prevented	Pass / to be verified in final implementation
T3	Assign one room to two classes at the same time	Conflict detected or assignment prevented	Pass / to be verified in final implementation
T4	Assign two classes to one student group in one period	Conflict detected or assignment prevented	Pass / to be verified in final implementation
T5	Submit incomplete input	Clear validation message	Pass / to be verified in final implementation
T6	Change a scheduling constraint after generation	Timetable regenerates using updated constraint	Pass / to be verified in final implementation
T7	Request timetable view by teacher	Teacher-specific schedule displayed	Pass / to be verified in final implementation
T8	Request timetable view by day	Daily schedule displayed	Pass / to be verified in final implementation
T9	Download generated timetable	Output available for download	Pass / to be verified in final implementation

Table 5.3: Test cases and expected behaviour

The status wording above is intentionally conservative: the project presentation describes these functions as system capabilities, but it does not provide a formal test log with measured pass/fail results. The final implementation should replace the verification notes with actual test observations.

5.5 Regeneration and Error Handling

The project identifies four major implementation challenges. Conflicting constraints are addressed by separating hard rules from soft preferences and scoring candidate timetables. Slow results on large inputs are addressed by using an efficient search and limiting the options checked. Messy or incomplete input is handled through form validation and clear error messages. Last-minute changes are handled through quick regeneration rather than rebuilding the entire timetable manually.

6. RESULTS AND DISCUSSION

6.1 Evaluation Setup

The evaluation of the proposed system focuses on whether the project objectives are addressed by the workflow and website features described in the presentation. Since the supplied project material does not include benchmark timings, a measured dataset, accuracy percentages or a recorded test log, this chapter reports functional and qualitative results rather than inventing numerical performance values.

6.2 Functional Results

Evaluation Area	Project Result / Expected Behaviour
Automatic generation	The system is designed to generate a timetable from the supplied scheduling inputs.
Teacher conflicts	The constraint model prevents or detects teacher double-booking.
Room conflicts	The system checks that rooms are not assigned simultaneously to multiple classes.
Student-group conflicts	The timetable is checked for overlapping classes for the same student group.
Weekly balance	Soft constraints encourage subjects and free periods to be distributed across the week.
Website access	Users can view the schedule by class, teacher or day.
Regeneration	Changes in inputs or rules can trigger rapid timetable regeneration.
Input validation	Incomplete or messy input is handled with form validation and clear error messages.
Export	The website concept includes download of the generated timetable.

Table 6.1: Functional evaluation summary

6.3 Interpretation of Results

The main strength of the proposed system is the combination of feasibility checking and schedule quality. Preventing clashes is treated as the first requirement, while workload distribution and practical spacing are treated as preferences. This is important because a timetable can be technically complete but still inconvenient for students or teachers if all demanding classes are concentrated on the same day.

The website also improves usability compared with manual spreadsheet editing. Instead of rebuilding a timetable after a change, the user can update an input or constraint and regenerate the schedule. The presentation explicitly identifies this automatic regeneration as a feature of the proposed system.

Another important result is the clear mapping between the AI component and the practical problem. The AI is not used merely for prediction; it is used to search for an arrangement that satisfies constraints. This makes the system a useful example of AI-based optimization in an educational administration context.

6.4 Comparison with the Objectives

Objective	Evidence from Project	Assessment
Automate timetable generation	AI engine generates the timetable from scheduling inputs.	Addressed
Avoid clashes	Hard constraints include teacher, room and class/student-group overlap prevention.	Addressed
Save time	Project goal is to replace days of manual scheduling with fast automated generation.	Addressed conceptually; no measured timing supplied
Balance workload	Soft rules distribute subjects and free periods and avoid overloaded days.	Addressed
Flexible updates	System regenerates when an input or rule changes.	Addressed
Easy access	Website supports entry, viewing, editing, regeneration and download.	Addressed

Table 6.2: Objective-wise result comparison

7. CONCLUSION AND FUTURE SCOPE

7.1 Summary

The AI-Based Student Timetable Generator addresses the practical limitations of manual timetable preparation by modelling scheduling as a constraint-optimization problem. The system accepts subjects, teachers, rooms, time slots and student groups, separates hard constraints from soft preferences, searches for a suitable arrangement, validates the result and presents it through a website.

The project focuses on preventing teacher, room and student-group clashes while improving the distribution of classes across the week. The proposed website adds usability through timetable generation, multiple viewing modes, editing, regeneration and downloading. These features directly reflect the supplied project presentation.

7.2 Learning Outcomes

- Understanding timetable generation as an AI-based constraint-optimization problem.
- Learning how hard constraints and soft preferences can be separated during system design.
- Understanding the role of search and scoring in selecting among multiple candidate schedules.
- Practising modular software design by separating input, scheduling, validation and presentation.
- Understanding the importance of validation and error handling in scheduling systems.
- Learning how a web interface can make an AI-based optimization system easier for non-technical users.
- Developing teamwork, documentation and presentation skills through a project-based learning process.

7.3 Future Enhancements

1. Electives and multi-section support so that the generator can handle more complex academic structures.
2. Mobile application support with notifications for students and staff.
3. Learning from previous preferences so that frequently preferred scheduling patterns can influence future results.
4. Support for more detailed room capabilities such as specialized laboratories and equipment requirements.
5. Role-based access so administrators, teachers and students see the functions relevant to them.
6. Historical timetable comparison to evaluate whether workload balance improves across semesters.
7. More advanced optimization and hybrid search methods for larger institutions.

8. Calendar integration so generated timetables can be synchronized with personal or institutional calendars.

The first three enhancements above are explicitly identified in the project presentation as future scope: electives and multi-section support, a mobile application with notifications, and learning from past preferences.

REFERENCES

- [1] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Pearson, 2020.
- [2] S. Schaerf, “A survey of automated timetabling,” Artificial Intelligence Review, vol. 13, pp. 87–127, 1999.
- [3] M. W. Carter and G. Laporte, “Recent developments in practical examination timetabling,” in Practice and Theory of Automated Timetabling, 1996.
- [4] E. K. Burke, P. De Causmaecker, G. Vanden Berghe, and H. Van Landeghem, “The state of the art of nurse rostering,” Journal of Scheduling, 2004.
- [5] E. K. Burke and S. Petrovic, “Recent research directions in automated timetabling,” European Journal of Operational Research, vol. 140, pp. 266–280, 2002.
- [6] C. L. Blake and J. W. Carter, “A goal programming approach to course timetabling,” European Journal of Operational Research, 1990s.
- [7] M. E. A. El-Sherbiny and M. A. M. Al-Sayed, “A genetic algorithm for university course timetabling,” related work in evolutionary scheduling literature.
- [8] Google OR-Tools, “Constraint Optimization and Scheduling,” technical documentation.
- [9] The supplied project presentation, “AI-Based Student Timetable Generator,” Woxsen University project material, 2026.